

Programming in Modern C++: Assignment Week 4

Total Marks : 20

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology
Kharagpur – 721302
partha.p.das@gmail.com

August 4, 2025

Question 1

Consider the below code segment.

[MCQ, Marks 2]

```
#include<iostream>
using namespace std;

class StatData{
    static int s;
public:
    void update(){ s = s * 2; }
    void print(){ cout << s; }
};

int StatData::s = 5;
int main(){
    StatData ob1, ob2;
    ob1.update();
    ob2.update();
    ob2.print();
    return 0;
}
```

What will be the output?

- a) 5
- b) 10
- c) 20
- d) 25

Answer: c)

Explanation:

`s` is a static member shared among all instances. Initially `s = 5`. The first call to `update()` doubles it to 10, and the second call doubles it again to 20.

Question 2

Consider the following code segment.

[MCQ, Marks 2]

```
#include<iostream>
using namespace std;

class Counter{
    static int count;
public:
    void increase(){
        count++;
    }
    ----- void display(){      //LINE-1
        cout << count << " ";
    }
};
int Counter::count = 0;

int main(){
    Counter c1, c2;
    c1.increase();
    c2.increase();
    Counter::display();
    return 0;
}
```

Fill in the blank at LINE-1 such that the program prints 2.

- a) virtual
- b) static
- c) friend
- d) mutable

Answer: b)

Explanation:

`count` is a static data member. To access `count`, `display()` must be declared as **static**. Again, `display()` can access static data without creating an object.

Question 3

Consider the following code segment.

[MSQ, Marks 2]

```
#include<iostream>
using namespace std;

class Box{
    int length;
public:
    Box(int l=1) : length(l) {}
    ----- //LINE-1
};

void printLength(Box &b){
    cout << b.length;
}

int main(){
    Box b(7);
    printLength(b);
    return 0;
}
```

Fill in the blank at LINE-1 such that the program will print 7.

- a) static void printLength(Box&);
- b) friend void printLength(Box&);
- c) void friend printLength(Box&);
- d) void static printLength(Box&);

Answer: b), c)

Explanation:

The global function `printLength()` accesses private member `length`, hence it must be a friend of `Box`. Both b) and c) are syntactically valid.

Question 4

Consider the following code segment.

[MCQ, Marks 2]

```
#include<iostream>
#include<cstring>
using namespace std;

class Message{
    char *text;
public:
    Message(const char* t): text(strdup(t)) {}
    Message operator+(const Message& m){ //LINE-1
        char* temp = new char[strlen(text) + strlen(m.text) + 1];
        strcpy(temp, text);
        strcat(temp, m.text);
        return Message(temp);
    }
    void print(){
        cout << text;
    }
};

int main(){
    Message m1("Hi "), m2("there!"); // LINE-1
    Message m3 = m1 + m2;
    m3.print();
    return 0;
}
```

What will be the output?

- a) Hithere!
- b) Hi there!
- c) Hi
- d) there!

Answer: b)

Explanation:

Overloaded operator+ concatenates two strings. m1 + m2 results in "Hi there!". The space between "Hi" and "there" results from creating the object m1, Hi with a space (refer LINE-1).

Question 5

Consider the below code segment.

[MSQ, Marks 2]

```
#include<iostream>

int main(){
    char name[] = "World";
    cout << name;    //LINE-1
    return 0;
}
```

The cout statement at LINE-1 gives an error. Change the cout statement such that it prints World.

- a) `std::cout << name;`
- b) `use cout << name;`
- c) `using std::cout; cout << name;`
- d) `std.name << name;`

Answer: a), c)

Explanation:

You need to access cout from std namespace. Either fully qualify or bring it into scope using using std.

Question 6

Consider the code segment below.

[MSQ, Marks 2]

```
#include<iostream>
using namespace std;

namespace physics{
    int speed = 100;
    int timing = 2;
}

namespace math{
    int speed = 5;
    int distance = 20;
}

using namespace physics;
int main(){
    cout << speed << endl;          //LINE-1
    cout << physics::timing << endl; //LINE-2
    cout << math::distance << endl; //LINE-3
    cout << timing << endl;          //LINE-4
    cout << math::speed << endl; //LINE-5
    return 0;
}
```

Which line(s) will give an error?

- a) LINE-1
- b) LINE-4
- c) LINE-3
- d) None will give an error

Answer: d)

Note: This is intentionally an MSQ.

Explanation:

LINE-2, 3, 5 are fully qualified. So, it will not give any error. LINE-1 and LINE-4 do not generate any error due to the statement using namespace physics.

Question 7

Consider the code segment below.

[MCQ, Marks 2]

```
#include<iostream>
using namespace std;

int x = 10;

namespace scopeTest{
    int x = 20;
}

int main(){
    int x = 5;
    cout << x + ::x + scopeTest::x << endl; //LINE-1
    return 0;
}
```

What will be the output?

- a) 35
- b) 15
- c) 25
- d) 30

Answer: a)

Explanation:

Local `x = 5` and `::x` refers to global `x (= 10)`, and `scopeTest::x = 20`. So, `Sum = 5 + 10 + 20 = 35`.

Question 8

Consider the code segment below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;
class Config{
    int value;
public:
    Config(int v) : value(v) {}
    int get() { return value; }
};
class System{
    static Config config;
public:
    static int getConfig(){
        return config.get();
    }
};
Config System::config(100);

int main(){
    cout << System::getConfig();
    return 0;
}
```

What will be the output?

- a) 0
- b) 100
- c) Compilation Error
- d) Linker Error

Answer: b)

Explanation:

System::getConfig() is a static function. It calls config.get(). config is already initialized with value = 100 by the statement Config System::config(100). So it returns 100, and prints it.

Question 9

Consider the code segment below.

[MCQ, Marks 2]

```
#include<iostream>
using namespace std;
class Secret{
    int code = 42;
    friend class Hacker;           //LINE-1
};
class Hacker{
public:
    void reveal(Secret s){
        cout << s.code;
    }
};
int main(){
    Secret s;
    Hacker h;
    h.reveal(s);
    return 0;
}
```

What will be the output/error?

- a) 42
- b) Compilation Error
- c) Linker Error
- d) Garbage Value

Answer: a)

Explanation:

The class `Hacker` is declared as a friend of `Secret`, that is, all member functions of `Hacker` can access even private members of `Secret`. In `main()`, a `Secret` object `s` and a `Hacker` object `h` are created. Calling `h.reveal(s)` prints `s.code` (which is 42). `reveal(s)` directly accesses `s.code`, which is private. This works because `Hacker` is a friend of `Secret`.

Programming Question 1

Consider the following program. Fill in the blanks according to the following instructions so that it satisfies all the test cases:

- at LINE-1 to allow the unary operator minus (-) to access private members,
- at LINE-2 with correct return statement.

Marks: 3

```
#include<iostream>
using namespace std;

class Complex{
    int re, im;
public:
    Complex(int r, int i) : re(r), im(i){}
    void print(){
        if (im >= 0)
            cout << re << " + j" << im;
        else
            cout << re << " - j" << -im;
    }
    ----- //LINE-1
};

Complex operator-(Complex c){
    return Complex(-----); //LINE-2
}

int main(){
    int a, b;
    cin >> a >> b;
    Complex c(a, b);
    Complex neg = -c;
    neg.print();
    return 0;
}
```

Public Test Case

Input: 5 3

Output: -5 - j3

Public Test Case

Input: 4 -2

Output: -4 + j2

Private Test Case

Input: -4 -2

Output: 4 + j2

Answer:

LINE-1: `friend Complex operator-(Complex c);`

LINE-2: `-c.re, -c.im`

Explanation:

The unary minus operator is overloaded to negate both real and imaginary parts. It requires access to private members. Therefore, at LINE-1 it is declared as friend of class `Complex` as:

`friend Complex operator-(Complex c);`

At LINE-2 the complex number can be returned as:

`return Complex(-c.re, -c.im);`

Programming Question 2

Consider the following program. Fill in the blanks according to the following instructions so that it satisfies all the test cases:

- at LINE-1 to make the instance `static`,
- at LINE-2 to complete the singleton accessor,
- at LINE-3 to initialize `*logInstance` outside the class.

Marks: 3

```
#include<iostream>
using namespace std;

class Logger{
    int count;
    _____ Logger *logInstance;           //LINE-1
    Logger(int c) : count(c) {}
public:
    int getCount(){ return count; }
    _____ getInstance(int v){           //LINE-2
        if(!logInstance)
            logInstance = new Logger(v);
        return logInstance;
    }
};

_____ Logger::logInstance = 0;             //LINE-3

int main(){
    int n; cin >> n;
    for(int i=0; i<n; i++){
        Logger *l = Logger::getInstance(i);
        cout << l->getCount() << " ";
    }
    return 0;
}
```

Public test Case

Input: 3

Output: 0 0 0

Public test Case

Input: 2

Output: 0 0

Private test Case

Input: 4

Output: 0 0 0 0

Answer:

```
LINE-1: static  
LINE-2: static Logger*  
LINE-3: Logger*
```

Explanation:

A singleton class ensures that only one object can be created for the class and it has to be reused. At LINE-1, a pointer to `Logger` object is defined as static as:

```
static Logger *logInstance;
```

At LINE-2, the header of the accessor to return instance of `Logger` can be defined as:

```
static Logger* getInstance(int v)
```

At LINE-3, the static class member `logInstance` can be initialized as:

```
Logger* Logger::logInstance = 0;
```

In main, within for loop, though the statement `Logger *l = Logger::getInstance(i);` is invoked; however, since `Logger` is a singleton class only one instance of the class is created. As a result, for all the iterations `getCount` returns the same value `c = 0` (which was the value of `i` during creation of `Logger` instance).

Programming Question 3

Consider the following program. Fill in the blanks according to the following instructions so that it satisfies all the test cases:

- at LINE-1 and LINE-2 to declare the overload functions for `operator>>` and `operator<<` as friend of class `Point`,
- at LINE-3 to define the function header for overloading `operator>>`,
- at LINE-4 to define function header for overloading `operator<<`

Marks: 3

```
#include<iostream>
using namespace std;

class Point{
    int x, y;
public:
    Point(int _x=0, int _y=0): x(_x), y(_y) {}

    _____; //LINE-1
    _____; //LINE-2
};

_____ (istream& is, Point& p){ //LINE-3
    is >> p.x >> p.y;
    return is;
}

_____ (ostream& os, const Point& p){ //LINE-4
    os << "(" << p.x << "," << p.y << ")";
    return os;
}

int main(){
    Point p;
    cin >> p;
    cout << p;
    return 0;
}
```

Public Test Case

Input: 3 4

Output: (3,4)

Public Test Case

Input: -5 8

Output: (-5,8)

Private Test Case

Input: -3 -4

Output: (-3,-4)

Answer:

```
LINE-1: friend istream& operator>>(istream&, Point&);  
LINE-2: friend ostream& operator<<(ostream&, const Point&);  
LINE-3: istream& operator>>  
LINE-4: ostream& operator<<
```

Explanation:

At LINE-3 and LINE-4 the stream insertion operator << and stream extraction operator >> are overloaded, which require the functions' header to be defined as follows:

```
istream& operator>>(istream& is, Point& p)  
ostream& operator<<(ostream& os, const Point& p)
```

At LINE-1 and LINE-2, these functions need to be defined as the friend of class `Point` as:

```
friend istream& operator>>(istream&, Point&);  
friend ostream& operator<<(ostream&, const Point&);
```